

Les Moniteurs

1. Définition

Un moniteur est un type de donnée abstrait qui possède des mécanismes de synchronisation et dont l'accès se réalise en exclusion mutuelle. Il comprend un ensemble de variables et un ensemble de procédures qui manipulent ces variables. Certaines procédures sont accessibles depuis l'extérieur et sont les points d'entrée du moniteur.

Les processus qui font appel à un moniteur pour se synchroniser n'ont pas un accès direct aux variables contenues dans le moniteur. Ils ne peuvent les atteindre que par les points processus est actif dans le moniteur, en train d'exécuter une de ses procédures.

Un moniteur contient aussi une partie d'initialisation qui n'est exécutée qu'une fois lors de la création du moniteur.

Pour résumé, les caractéristiques principales du moniteur sont :

- Les variables locales sont accessibles uniquement par les procédures du moniteur.
- Un processus entre dans le moniteur en appelant une des procédures du moniteur
- Un seul processus peut exécuter le moniteur (être actif dans le moniteur) à un instant donné ; tout autre processus qui appelle le moniteur est suspendu jusqu'à ce que le moniteur devient libre.

2. Structure de Moniteur

```
Monitor Nom moniteur { // début de moniteur
    variables locales
    variable de condition

    void procedure 1( ) { }
    void procedure N( ){ }
    {
        // code initialisation
    }
} // fin de moniteur
```

3. Les variables de condition

Un moniteur permet de faire la synchronisation en utilisant des variables spéciales appelées les variables de condition qui sont déclarées dans le moniteur et accessible uniquement à l'intérieur du moniteur.

La déclaration d'une variable de condition peut se faire comme suit :

condition c ;

Deux fonctions primitives opèrent sur les variables de condition :

- **cwait(c)** : suspend l'exécution d'un processus appelant sur la condition c. le moniteur devient disponible pour un autre processus.
- **csignal(c)** : réveille un processus suspendu après un cwait sur la même condition. Si plusieurs processus sont suspendus sur la condition c, alors choisir l'un entre eux, si aucun processus n'est suspendu sur c, alors ne rien faire.

Remarque :

Les opérations cwait et csignal sont différents de leurs équivalentes dans les sémaphores :

- Si un processus exécute un csignal sur une condition c et qu'aucun processus n'est suspendu sur la condition c alors ce signal est perdu.
- Un processus qui exécute un cwait sur une condition c est systematiquement suspendu.

Exemple : la synchronisation de problème de producteur/consommateur par le moniteur

```
#define N 100 // taille de tampon
```

```
Monitor Prod_Cons { // début de moniteur
char buffer[N] ;
int in, out ; // indices d'entrée et de sortie du buffer
int count; // nombre d'objets dans le buffer
condition vide, plein;

void deposer (char x) { //déposer un objet x dans le buffer
if (count == N) cwait(vide) ;
buffer[in] = x;
in = (in + 1) % N;
count ++;
csignal(plein);
}

void retirer (char *x) { //retirer un objet de buffer
if (count == 0) cwait(plein) ;
*x = buffer[out];
out = (out +1) % N;
count --;
csignal(vide);
}
```

```

{ // code initialisation
    in = 0;
    out = 0;
    count = 0;
}

} // fin de moniteur

void producteur ( ) { //processus producteur
    char x ;
    while(1){
        produire(&x) ;
        Prod_Cons.deposer(x) ;
    }
}

void consommateur ( ) { // processus consommateur
    char x ;
    while(1){
        Prod_Cons.retier(&x) ;
        consommer (x) ;
    }
}

int main ( ) {
    parbegin(producteur(), consommateur() ) ;
    return 0 ;
}

```